

Министерство высшей науки и образования РФ

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4 по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнил: ст. гр. 25BBB2

Тургенев И.В.

Козлов А.В.

Проверил: к.т.н. доцент

Юрова О.В.

к.т.н. доцент

Деев М.В.

Пенза 2026

Название

Бинарное дерево поиска

Цель работы:

1. Создать бинарное дерево поиска (БДП) — структуру данных, где каждый узел имеет не более двух «детей» (left и right)
2. Правило дерева: **влево — меньшие значения, вправо — большие**

Лабораторное задание:

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
- 4* Оценить сложность процедуры поиска по значению в бинарном дереве.

Описание использованных методов для решения заданий

Задание 1:

Структура данных

C

```
struct Node {  
    int data;          // значение узла  
    struct Node* left; // левое поддерево  
    struct Node* right; // правое поддерево  
};  
struct Node* root = NULL; // глобальный корень
```

Разбор функций

CreateTree(root, r, data) — вставка элемента

Функция рекурсивная, принимает два указателя:

- root — родительский узел текущего узла,
- r — текущий рассматриваемый узел (поддерево),

- data — вставляемое значение.

Алгоритм:

1. Если $r == \text{NULL}$ — нашли свободное место: выделяется память под новый узел (malloc), в случае неудачи выводится сообщение и `exit(0)`. Поля left/right обнуляются, записывается data.
 - Если $r == \text{NULL}$ — дерево было пустым, новый узел возвращается и становится корнем.
 - В противном случае узел подвешивается к родительскому узлу: $\text{data} > r \rightarrow \text{data} \rightarrow$ в левое поле родительского узла, иначе $\rightarrow$ в правое.
2. Если $r \neq \text{NULL}$ — спуск вниз: при $\text{data} > r \rightarrow \text{data}$ идём в $r \rightarrow \text{left}$, иначе в $r \rightarrow \text{right}$. Результат рекурсивного вызова игнорируется (он и не нужен — привязка происходит внутри), возвращается root.

`print_tree(r, l)` — вывод дерева

Обратный симметричный обход (правое $\rightarrow$ узел $\rightarrow$ левое) с отступом в l пробелов, где l — глубина узла. В результате дерево печатается повернутым на 90° против часовой стрелки: корень — слева у края, потомки — со сдвигом вправо. Поскольку «меньшие» числа находятся справа, значения выводятся сверху вниз по возрастанию.

`SearchTree(r, data)` — задание 1, поиск

Рекурсивный поиск, полностью соответствующий правилу вставки:

- $r == \text{NULL} \rightarrow$ возвращает `NULL` (не найдено);
- $\text{data} == r \rightarrow \text{data} \rightarrow$ возвращает указатель на узел;
- $\text{data} > r \rightarrow \text{data} \rightarrow$ ищем в left, иначе в right.

Сложность — $O(h)$, где h — высота дерева ($O(\log n)$ для сбалансированного дерева, $O(n)$ в худшем случае — при вводе упорядоченной последовательности дерево вырождается в список).

main

1. `setlocale, SetConsoleCP(1251), SetConsoleOutputCP(1251)` — настройка консоли на кириллицу (MSVC/Windows).

2. Цикл ввода чисел: каждое число вставляется в дерево, -1 завершает построение (т. е. -1 нельзя хранить в дереве).
3. `print_tree(root, 0)` — печать дерева.
4. Запрос числа для поиска, вызов `SearchTree`, вывод «найдено / не найдено».
5. Последний `scanf_s("%d", &D)` — «заглушка», чтобы окно консоли не закрывалось сразу.

Задание 2:

`CreateTree(root, r, data)` — вставка

Рекурсивная вставка с передачей пары «родитель (root) — текущий узел (r)»:

1. Если `r == NULL` — место найдено:
 - `malloc` под новый узел, при неудаче — сообщение и `exit(0)`;
 - поля `left`, `right` = `NULL`, `data` = введенное значение;
 - если `root == NULL`, дерево было пустым → узел возвращается и становится корнем;
 - в противном случае узел прикрепляется к родительскому узлу: `data > root->data` → в **левое** поле, иначе (меньше **или равно**) → в **правое**.
2. В противном случае происходит спуск: `data > r->data` → в `r->left`, иначе → в `r->right`; возвращается `root`.

`print_tree(r, l)` — **печать**

Обход «правое → корень → левое» с отступом в `l` пробелов. Дерево выводится боком (корень у левого края); из-за инвертированного порядка значения печатаются сверху вниз по возрастанию.

`CountOccurrences(r, data)` — задание 2

C

```
if (r == NULL) return 0;
```

```
int count = (r->data == data) ? 1 : 0;
```

```
count += CountOccurrences(r->left, data);
```

```
count += CountOccurrences(r->right, data);
```

return count;

Это **полный обход всего дерева** (по сути, прямой обход): в каждом узле проверяется равенство, к результату прибавляются счетчики обоих поддеревьев. Возвращается общее количество совпадений.

- Сложность — $O(n)$, обходятся все узлы независимо от значения.
- Плюс подхода: работает корректно всегда, даже если бы упорядоченность дерева была нарушена.
- Минус: не использует свойство дерева поиска. Поскольку дубликаты всегда идут в правую ветвь, можно было ограничиться спуском по дереву:

Это даст $O(h + k)$ вместо $O(n)$.

main

1. Настройка локали и кодовых страниц консоли.
2. Цикл ввода: числа вставляются в дерево, **-1 завершает ввод** (то есть -1 в дерево поместить нельзя).
3. `print_tree(root, 0)` — вывод построенного дерева.
4. Запрос числа и вывод результата `CountOccurrences` (0, если числа нет).
5. Финальный `scanf_s` — задержка перед закрытием окна консоли.

Задание 3:

`CreateTreeNoDup(root, r, data)` — вставка без дубликатов

Параметры: `root` — родительский узел текущего узла, `r` — текущее поддерево, `data` — вставляемое значение.

Логика по шагам:

1. `r == NULL` — свободное место найдено:
 - `malloc` под узел; в случае неудачи — сообщение об ошибке и `exit(0)`;
 - `left = right = NULL, data = data`;
 - если `root == NULL` — дерево было пустым, узел возвращается и становится корнем;
 - иначе узел подвешивается к родительскому узлу: `data > root-`

>data → в левое поле, иначе → в правое.

2. data == r->data — новая ветка по сравнению с предыдущими вариантами: выводится

«Значение %d уже существует в дереве. Добавление отменено.» и функция завершается, не создавая узел. Возвращается root (на верхнем уровне это настоящий корень, во вложенных вызовах результат все равно игнорируется).

3. В противном случае — рекурсивный спуск: data > r->data → в r->left, иначе → в r->right; в конце возвращается root.

print_tree(r, l)

Обход «правое поддерево → узел → левое поддерево» с отступом в l пробелов (l — глубина). Дерево печатается повёрнутым на 90°: корень у левого края, потомки смещены вправо. Поскольку меньшие значения находятся справа, числа идут по возрастанию сверху вниз.

main

1. Настройка локали и кодовых страниц консоли.
2. Цикл ввода чисел; каждое число передаётся в CreateTreeNoDup. Ввод -1 завершает построение (следовательно, -1 хранить в дереве нельзя).
3. print_tree(root, 0) — вывод дерева; каждое значение встречается ровно один раз.
4. Финальный scanf_s — «пауза», чтобы окно консоли не закрылось.

Пример: Ввод 5 3 8 5 3 -1 → дважды выводится сообщение об отмене, в дереве остаются только числа 5, 3, 8.

Задание 4:

CreateTree(root, r, data) — вставка

Рекурсия с парой указателей «родитель root — текущий узел r»:

1. r == NULL — место найдено: malloc (в случае неудачи — сообщение и exit(0)), поля left/right обнуляются, записывается data.
 - если root == NULL — дерево пустое, узел возвращается и становится корнем;

- в противном случае он подвешивается к родительскому узлу: `data > root->data` → **левое** поле, иначе → **правое**.

2. Иначе спуск: `data > r->data` → в `r->left`, иначе → в `r->right`; возвращается `root`.

Особенности: порядок **инвертирован** (большие значения — влево, меньшие/равные — вправо); дубликаты допускаются и уходят вправо.

`print_tree(r, l)`

Обход «правое → узел → левое» с отступом в `l` пробелов: дерево печатается повернутым на 90°, значения идут сверху вниз по возрастанию.

`SearchTree(r, data, steps)` — **поиск с подсчетом сравнений**

Отличие от задания 1 — третий параметр `int* steps`, указатель на счетчик:

C

```
if (r == NULL) return NULL; // дошли до "пустоты" — не найдено
```

```
(*steps)++; // посетили узел => одно сравнение
```

```
if (data == r->data) return r;
```

```
if (data > r->data) return SearchTree(r->left, data, steps);
```

```
return SearchTree(r->right, data, steps);
```

- Счетчик увеличивается в **каждом посещенном узле**, поэтому итоговое значение = длине пройденного пути от корня (количеству просмотренных уровней).
- Передача через указатель нужна для того, чтобы изменения были видны в `main` (в языке C нет передачи по ссылке).
- При неудачном поиске счетчик показывает глубину, на которой был обнаружен `NULL`.
- Направления спуска согласованы с правилом вставки, поэтому поиск корректен.

main

1. Настройка локали и кодовых страниц.
2. Цикл ввода чисел, **-1 завершающий построение** (-1 в дерево не попадает).

3. Печать дерева.
4. Ввод искомого значения, обнуление steps, вызов SearchTree(root, D, &steps), вывод результата и количества сравнений.
5. Вывод справки о сложности:
 - число сравнений $\leq$ высоты дерева h ;
 - сбалансированное дерево $\rightarrow O(\log n)$;
 - вырожденное (цепочка, например при вводе отсортированной последовательности) $\rightarrow O(n)$.
6. Финальный scanf_s — задержка перед закрытием окна.

Пример. Ввод 5 3 8 2 4 -1, поиск 4: путь $5 \rightarrow 3 \rightarrow 4$, счетчик = 3.

Результаты работы программы

Задание 1:

```
1 - режим построения
Введите число: 12
Введите число: 38
Введите число: 428
Введите число: 92
Введите число: 912
Введите число: 326
Введите число: 17
Введите число: 19
Введите число: 237
Введите число: 235
Введите число: 29
Введите число: 14
Введите число: 115
Введите число: 34
Введите число: 64
Введите число: -1
Построение дерева окончено
```

```
12
  14
  17
  19
  29
  34
38
  64
  92
    115
    235
    237
    326
428
  912
```

```
Введите значение для поиска: 12
Значение 12 найдено в дереве.
```

Задание 2:

```

Введите число: 347
Введите число: 37
Введите число: 19
Введите число: 324
Введите число: 19
Введите число: 24
Введите число: 34
Введите число: 317
Введите число: 94
Введите число: 43
Введите число: 93
Введите число: 47
Введите число: -1
Построение дерева окончено

```

```

13
  19
    19
      19
        24
          28
            34
              37
                43
                  47
                    53
                      93
                        93
                          94
                            317
                              324
                                347

```

```

Введите значение для подсчёта вхождений: 19
Значение 19 встречается в дереве 3 раз(а).

```

Задание 3:

```

3 - Изменить добавление, чтобы не добавлялись одинаковые значения
Введите число: 29
Введите число: 18
Введите число: 27
Введите число: 347
Введите число: 237
Введите число: 348
Введите число: 943
Введите число: 84
Введите число: 38
Введите число: 74
Введите число: 219
Введите число: 28
Введите число: 29
Значение 29 уже существует в дереве. Добавление отменено.
Введите число: -1
Построение дерева окончено

```

```

18
  27
    28
      29
        38
          74
            84
              219
                237
                  347
                    348
                      943

```

Задание 4:

```
4 – Оценить сложность поиска по значению в бинарном дереве
Введите число: 328
Введите число: 327
Введите число: 17
Введите число: 243
Введите число: 34
Введите число: 28
Введите число: 19
Введите число: 347
Введите число: 346
Введите число: 72
Введите число: 91
Введите число: 03
Введите число: 32
Введите число: 43
Введите число: 59
Введите число: -1
Построение дерева окончено

      3
     17
      19
      28
      32
     34
      43
      59
      72
      91
     243
    327
   328
    346
    347

Введите значение для поиска: 328
Значение 328 найдено. Число сравнений: 1

Оценка сложности поиска в бинарном дереве:
Число сравнений равно высоте дерева h.
Сбалансированное дерево:  $O(\log n)$ .
Вырожденное дерево (цепочка):  $O(n)$ .
```

Выводы

В ходе выполнения лабораторной работы было изучено и реализовано бинарное дерево-структура данных, где каждый узел имеет одного из «детей».